POSTGRESQL

Disaster Recovery with pgBackRest TLS Transport

Production Technical SOP & Recovery Runbook

Secure backup-repository access, restore validation, PITR, isolation controls and operational recovery procedures

Document Control	Value
Document Type	Technical SOP / Disaster Recovery Runbook
Technology	PostgreSQL + pgBackRest
Transport	Mutual TLS (X.509)
Scope	Backup repository to isolated DR restore host
Validation Basis	Stormatics article (**Shridhar Khanal**), pgBackRest documentation, PostgreSQL official documentation
Prepared	30 August 2026
Change Safety	Test in non-production; adapt paths, versions, PKI and firewall rules to your environment

Important: The commands in this SOP use example names and PostgreSQL 15-style RHEL paths where useful. Replace all hostnames, paths, stanza names, certificate identities and PostgreSQL versions with values from your environment.

Table of Contents

1. Purpose and Scope

This SOP defines a controlled method to restore PostgreSQL from a pgBackRest repository to an isolated Disaster Recovery (DR) server using pgBackRest native TLS transport rather than SSH. The design uses certificate-based mutual authentication and stanza-level authorization.

- Restore a full PostgreSQL cluster from a pgBackRest repository.
- Perform latest-backup recovery or Point-in-Time Recovery (PITR).
- Keep the DR host isolated from shell access to the backup server.
- Validate backup availability before a recovery event.
- Prevent a restored DR instance from accidentally writing WAL into the production archive.
- Provide repeatable validation, troubleshooting, security and rollback steps.

This SOP is not a substitute for organization-specific DR policy. Recovery targets, DNS/application cutover, network routing and business approval must be integrated into the enterprise DR plan.

2. Architecture and Recovery Flow

pgBackRest TLS server mode allows remote repository access without using the SSH protocol. The current pgBackRest documentation lists TLS as a supported repository-host protocol and uses TCP 8432 as the default TLS repository-host port.

3. Key Design Principles

Principle	Production Guidance
Isolation	The DR host should not require shell access to the backup node merely to restore pgBackRest data.
Mutual authentication	The client validates the repository server certificate and the server validates the DR client certificate.
Least privilege	Authorize the client certificate CN only for required pgBackRest stanza(s).
Repository protection	Do not let a test/DR instance archive WAL into the production repository unless the architecture explicitly requires and isolates it.
Recoverability	A backup is operationally useful only after restore tests prove it can meet RPO/RTO.
Version alignment	Install a PostgreSQL major version compatible with the backup being restored; validate extensions and OS dependencies.

Principle	Production Guidance
Evidence	Capture pgBackRest info output, restore logs, PostgreSQL startup logs and validation results for each drill.

4. Prerequisites and Pre-Change Checks

- Existing pgBackRest repository contains valid backups and WAL required for the intended recovery point.
- DR server has sufficient CPU, memory and storage for restored PGDATA, tablespaces, WAL replay and temporary recovery overhead.
- PostgreSQL binaries for the required major version are installed on the DR host.
- Compatible pgBackRest version is installed on the DR host and repository server.
- DNS or IP connectivity exists from DR to backup node.
- TCP 8432 (or your configured TLS server port) is allowed only from approved source networks.
- Time synchronization is healthy on both hosts; certificate validation depends on correct system time.
- Organization-approved CA and certificate lifecycle process are available.
- Tablespace mount points and permissions are recreated if the source cluster uses tablespaces.
- Application traffic is blocked from the DR database until validation and formal cutover approval.

```
# Capture versions
postgres --version
pgbackrest version
openssl version

# Confirm repository inventory on the repository host
sudo -u postgres pgbackrest --stanza=db info

# Check listening ports after TLS service is configured
ss -lntp | grep 8432
```

5. Security and PKI Design

For production, prefer an enterprise PKI or managed certificate authority. A locally created CA is useful for a lab, but introduces CA-key protection, renewal, expiry monitoring and distribution responsibilities.

Certificate / Key	Location	Purpose	Permission Guidance
CA certificate	Backup + DR	Trust anchor	Readable by pgBackRest service account
CA private key	Secure CA only	Signs certificates	Do not distribute to DR or routine backup service hosts
Server certificate	Backup node	Identifies TLS server	Readable by service account
Server private key	Backup node	Proves server identity	Restrict to service account
Client certificate	DR node	Identifies DR client	Readable by service account
Client private key	DR node	Proves client identity	Mode 0600 or equivalent

pgBackRest authorizes TLS clients using the certificate Common Name (CN) mapped to a stanza with tls-server-auth. Current pgBackRest documentation also states that the TLS server verifies client certificates against a trusted CA.

5.1 Example Lab Certificate Creation

The following is a lab-style example. In production, use your enterprise CA, certificate templates, SAN requirements, rotation standards and secret-management process.

```
# Example CA - LAB ONLY
sudo mkdir -p /etc/pgbackrest/certs
cd /etc/pgbackrest/certs
sudo openssl genrsa -out ca.key 4096
sudo openssl req -new -x509 -days 3650 -key ca.key \
  -subj "/CN=pgBackRest-CA" -out ca.crt
sudo chmod 600 ca.key
sudo chmod 644 ca.crt

# Backup-node server key and CSR
sudo openssl genrsa -out server.key 4096
sudo openssl req -new -key server.key \
  -subj "/CN=backup-node.example.com" -out server.csr

# DR client key and CSR
sudo openssl genrsa -out client.key 4096
sudo openssl req -new -key client.key \
  -subj "/CN=dr-server" -out client.csr
```

For modern production PKI, ensure server identity validation matches your pgBackRest/PKI requirements. Prefer certificates with appropriate Subject Alternative Names (SANs) when required by your certificate policy.

6. Prepare the DR Server

Install the same PostgreSQL major version needed by the backup. The package commands depend on the OS and repository. The Stormatics example uses RHEL 9 and PostgreSQL 15; do not hard-code that version for every recovery.

```
# Example only - RHEL-family host / PostgreSQL 15
sudo dnf install -y postgresql15-server
sudo dnf install -y pgbackrest

# Create restore target
sudo mkdir -p /var/lib/pgsql/15/restored-data
sudo chown postgres:postgres /var/lib/pgsql/15/restored-data
sudo chmod 700 /var/lib/pgsql/15/restored-data
```

- Do not initialize a new cluster into the final restore directory unless your restore procedure explicitly needs it.
- Ensure the target directory is empty before restore.
- Recreate required tablespace directories before starting the restore.
- Verify filesystem capacity and mount options.

7. Configure the pgBackRest TLS Server

On the backup/repository node, configure the pgBackRest server listener and certificate authorization. Example:

```
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2
log-level-console=info
log-level-file=info

tls-server-address=*
tls-server-port=8432
tls-server-cert-file=/etc/pgbackrest/certs/server.crt
tls-server-key-file=/etc/pgbackrest/certs/server.key
tls-server-ca-file=/etc/pgbackrest/certs/ca.crt

# Client certificate CN 'dr-server' may access stanza 'db'
tls-server-auth=dr-server=db

[db]
pg1-path=/var/lib/pgsql/15/data
```

Do not expose tls-server-address=* to untrusted networks. Enforce source restrictions with host firewall rules, security groups, network ACLs or equivalent controls.

7.1 Run the TLS Server as a Managed Service

```
# Example systemd unit
[Unit]
Description=pgBackRest TLS Server
After=network-online.target
Wants=network-online.target

[Service]
User=postgres
ExecStart=/usr/bin/pgbackrest server
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now pgbackrest
sudo systemctl status pgbackrest
ss -lntp | grep 8432
```

8. Configure the DR Client

```
[global]
repo1-host=backup-node.example.com
repo1-host-type=tls
repo1-host-port=8432
repo1-host-cert-file=/etc/pgbackrest/certs/client.crt
repo1-host-key-file=/etc/pgbackrest/certs/client.key
repo1-host-ca-file=/etc/pgbackrest/certs/ca.crt
log-level-console=info

[db]
pg1-path=/var/lib/pgsql/15/restored-data
```

The important difference from an SSH transport is repo1-host-type=tls plus certificate/key/CA configuration. pgBackRest's current configuration reference documents both ssh and tls repository-host protocol types.

9. Connectivity and Repository Validation

1. Confirm DNS resolution for the repository hostname.
2. Confirm TCP connectivity to the configured TLS port.
3. Validate the TLS certificate chain and identity.
4. Run pgBackRest info from the DR server.
5. Confirm the expected stanza, backup labels and WAL archive ranges are visible.

```
getent hosts backup-node.example.com
nc -zv backup-node.example.com 8432

# Repository visibility from DR
sudo -u postgres pgbackrest --stanza=db info
```

Do not begin the destructive part of a recovery until the required backup and recovery target are confirmed.

10. Restore the Latest Backup

Stop PostgreSQL on the DR target if it is running. Preserve any required forensic data, then ensure the restore target is empty and correctly owned.

```
sudo systemctl stop postgresql-15 2>/dev/null || true

sudo -u postgres pgbackrest --stanza=db \
  --pg1-path=/var/lib/pgsql/15/restored-data \
  --process-max=4 \
  --log-level-console=info \
  restore
```

Tune process-max according to CPU, storage throughput and network capacity. More parallelism is not automatically faster if the repository, network or target storage becomes saturated.

11. Point-in-Time Recovery (PITR)

PostgreSQL PITR restores a physical base backup and replays the required continuous WAL stream until a recovery target is reached. PostgreSQL official documentation stresses that successful PITR requires an unbroken sequence of WAL from the backup through the target recovery point.

```
sudo -u postgres pgbackrest --stanza=db \
  --pg1-path=/var/lib/pgsql/15/restored-data \
  --type=time \
  --target="2026-05-12 11:30:00" \
  --target-action=promote \
  --process-max=4 \
  --log-level-console=info \
  restore
```

Target Type	Typical Use
time	Recover to a known timestamp before a bad deployment or data change.
name	Recover to a named restore point created before a controlled change.
xid / LSN	Specialized recovery where a precise transaction or WAL position is known and validated.
immediate / latest behavior	Recover to the earliest consistent point or through available WAL, depending on selected pgBackRest/PostgreSQL recovery options.

Always record the requested target, timezone, backup label and business event that justifies the target. Timestamp mistakes are a common DR failure mode.

12. Start PostgreSQL Safely

Before starting an isolated DR/test instance, review the restored PostgreSQL configuration. A critical control is preventing the DR instance from pushing WAL into the live production archive unless this is intentionally designed.

```
# Review effective archive configuration before startup
grep -E '^[[:space:]]*(archive_mode|archive_command|archive_library)' \
  /var/lib/pgsql/15/restored-data/postgresql.conf

# For an isolated restore drill, disable production archiving as appropriate
# archive_mode = off
```

Also review primary_conninfo, primary_slot_name, restore/recovery settings, synchronous_standby_names, shared_preload_libraries, listen_addresses, port and any extension paths. Do not allow the restored host to accidentally connect to production replication or monitoring endpoints with write-capable credentials.

```
sudo -i -u postgres
/usr/pgsql-15/bin/pg_ctl \
  -D /var/lib/pgsql/15/restored-data \
  -l /var/lib/pgsql/15/dr-logfile start
```

13. Post-Restore Validation

```
psql -U postgres -d postgres -c "SELECT version();"
psql -U postgres -d postgres -c "SELECT pg_is_in_recovery();"
psql -U postgres -d postgres -c "SELECT now();"
psql -U postgres -d postgres -c "\l+"

# Check cluster/timeline-related logs
tail -100 /var/lib/pgsql/15/dr-logfile
```

- Confirm PostgreSQL starts without recovery errors.
- Confirm expected databases, schemas and critical objects exist.
- Validate the expected recovery point with business-owned data checks.
- Validate extensions and shared libraries.
- Validate tablespace mappings.
- Run application smoke tests in an isolated network.
- Check PostgreSQL logs for missing WAL, permission, checksum or extension errors.
- Record actual restore duration and achieved recovery point.

14. Failover / DR Activation Checklist

Step	Owner / Evidence
Declare DR event and freeze conflicting production changes	Incident Commander / ticket
Confirm source state and selected recovery target	DBA + application owner
Restore and validate PostgreSQL	DBA / restore log
Confirm production archive is protected from DR writes	DBA / configuration evidence
Validate data consistency and application smoke tests	DBA + application team
Approve application cutover	Business + incident commander
Change DNS / routing / secrets as designed	Platform/network team
Monitor database and application after cutover	DBA/SRE
Document actual RPO and RTO	Incident record

15. Failback Considerations

Failback is not simply the reverse of restore. Once applications write to the DR database, it becomes a new source of truth. Plan a controlled migration or replication path back to the preferred production site.

- Do not start the old primary and allow applications to write to both sites.
- Establish which cluster owns the authoritative timeline.
- Choose physical replication, logical replication, backup/restore, or another approved data movement method.
- Validate WAL timeline and replication compatibility.
- Perform a second controlled cutover with application quiescence or an approved near-zero-downtime method.
- Re-establish backup, archiving, monitoring and DR protection after failback.

16. Monitoring and Operational Controls

Control	Recommended Check
Backup status	pgbackrest --stanza=<name> info
Repository consistency	Run pgBackRest check according to your backup policy

Control	Recommended Check
Backup age	Alert when last successful full/diff/incr exceeds policy
WAL archive health	Alert on archive_command failures and WAL backlog
Certificate expiry	Alert well before server/client/CA certificate expiry
TLS listener	Monitor service status and TCP reachability
Repository capacity	Monitor filesystem/object-store usage and retention
Restore drill	Perform on a defined schedule and capture RTO/RPO evidence
Config drift	Version-control pgBackRest and systemd configuration without private keys

17. Troubleshooting

Symptom	Likely Cause	Action
TLS handshake failure	CA mismatch, expired cert, identity mismatch	Verify CA chain, dates, server/client certs and host identity.
Authorization denied	Client CN not mapped to stanza	Verify certificate CN and tls-server-auth mapping.
Connection refused / timeout	Service down or port blocked	Check systemd, ss, firewall and network ACL/security group.
Stanza not found	Config mismatch	Compare stanza name on DR and repository node.
Restore cannot find WAL	Archive gap or wrong recovery target	Check pgBackRest info, archive inventory and target time/timeline.
Permission denied	Wrong PGDATA/cert/repository ownership	Correct ownership and restrictive key permissions.
PostgreSQL will not start	Version, config, extension, tablespace or recovery issue	Inspect PostgreSQL log before changing files.
Unexpected repository writes	DR archiving still points to production repository	Stop DR instance and isolate archive settings immediately.

18. Security Hardening

- Use an enterprise CA when available; protect the CA private key outside routine database hosts.
- Use short, policy-driven certificate lifetimes and automated expiry monitoring.
- Restrict the TLS listener to approved DR networks; do not expose it broadly to the Internet.
- Authorize each client CN only to required stanza(s). Avoid broad wildcard stanza access unless justified.
- Store private keys with least-privilege filesystem permissions and protect backups of those keys.
- Run the pgBackRest TLS service under a dedicated least-privilege account where your architecture supports it.
- Separate production and DR/test repository write paths where possible.
- Keep pgBackRest and PostgreSQL on supported, patched versions.
- Centralize audit logs and alert on repeated TLS/authentication failures.
- Treat a compromised client certificate as an incident: remove its tls-server-auth authorization, restart/reload the service as required, and reissue identity under a new approved CN.

The referenced Stormatics article notes that pgBackRest does not provide CRL handling for this authorization model. Therefore, certificate revocation/compromise response must be explicitly designed around authorization removal, certificate replacement and PKI controls.

19. Restore Testing Program

A DR plan should be tested, not merely documented. Run scheduled recovery exercises using an isolated host or environment.

Test	Success Criteria
Repository access	DR client can authenticate over TLS and list intended stanza backups.
Latest restore	Cluster restores and starts successfully.
PITR	Cluster stops/promotes at the requested validated business point.
Data validation	Critical application objects and sample business transactions match expectations.
Isolation	No unintended production WAL archive, replication or application connections occur.
Performance	Restore completes within target RTO or produces an improvement action plan.
Evidence	Logs, timings, target point, backup label and sign-off are retained.

20. Rollback and Abort Plan

Abort recovery or cutover when any critical prerequisite is not met: missing WAL, invalid backup, certificate failure, insufficient disk, wrong PostgreSQL major version, failed data validation or risk of repository contamination.

6. Stop PostgreSQL on the DR host.
7. Block application access to the DR endpoint.
8. Preserve restore and PostgreSQL logs.
9. Do not modify the production source as part of a failed restore test.
10. Correct the failed prerequisite.
11. Clean or recreate the DR restore directory according to policy.
12. Repeat the restore from a known state.
13. Escalate if the requested RPO/RTO cannot be met.

21. RPO/RTO Guidance

Metric	Meaning	pgBackRest / PostgreSQL Impact
RPO	Maximum acceptable data loss	Driven by WAL archival continuity/frequency and the available recovery target.
RTO	Maximum acceptable service restoration time	Driven by backup size, repository throughput, network, target storage, WAL replay and validation/cutover time.
Backup retention	How far back recovery can go	Must retain a usable base backup plus all WAL needed to reach the target.
Restore bandwidth	How quickly backup data can reach DR	Measure during drills; do not estimate only from link speed.
WAL replay time	Time to roll forward after base restore	Can dominate RTO when recovering from an older base backup with heavy WAL volume.

22. Production Checklist

☐ Approved DR ticket / incident declared
☐ Correct stanza and repository confirmed
☐ Correct PostgreSQL major version installed
☐ DR storage and tablespaces ready
☐ TLS certificates valid and not near expiry
☐ Port 8432 restricted and reachable
☐ pgBackRest info succeeds from DR
☐ Recovery target and timezone confirmed
☐ Restore command peer-reviewed
☐ Restore completed without fatal errors

☐ Production archive writes disabled for isolated DR test
☐ PostgreSQL startup validated
☐ Business data validation completed
☐ Application smoke test completed
☐ RPO/RTO measured
☐ Cutover approved
☐ Monitoring and backup strategy re-established after activation
☐ Evidence and lessons learned recorded

23. References

Primary article:

Stormatics - PostgreSQL Disaster Recovery with pgBackRest TLS Transport

https://stormatics.tech/blogs/postgresql-disaster-recovery-with-pgbackrest-tls-transport

Authoritative technical references:

pgBackRest Configuration Reference

https://pgbackrest.org/configuration.html

pgBackRest Command Reference

https://pgbackrest.org/command.html

PostgreSQL Official Documentation - Continuous Archiving and Point-in-Time Recovery (PITR)

https://www.postgresql.org/docs/current/continuous-archiving.html

PostgreSQL Official Documentation - Backup and Restore

https://www.postgresql.org/docs/current/backup.html

Operational note:

- PostgreSQL and pgBackRest features, parameter names and defaults can change.
- Validate the exact PostgreSQL major version and pgBackRest release used in your environment before implementation.